



Simply Rugby System

Solution Planning Document

Contents

1. Solution Planning Overview	3
• Purpose of the Report.....	3
• Project Overview.....	3
2. Business Model - Functional Analysis and Dynamic Design	3
• Identified Classes, Attributes and Methods.....	3
• Use Case Diagram	5
• Use Case Model 1 – "Login "	6
• Use Case Model 2 – "Edit Player Skills"	7
• Sequence Diagram	8
• Activity Diagram	9
3. Static Model (UML)	10
• Class Diagram	10
4. User and Interface Analysis.....	11
• User Analysis	11
• HCI Design Considerations	11
5. User Interface Design.....	12
• Wireframes	12
6. Planning Summary and Next Steps	13
• Planning Stage Completion	13
• Pending Design Items.....	13

1. Solution Planning Overview

- Purpose of the Report

The purpose of this report is to present the planning stage of the application development process for the Simply Rugby team. covers topics such as the analysis and design of the proposed system before the development phase. This stage concentrates on understanding the system functional requirements, modelling the application's structure and behaviour using UML, and designing an appropriate user interface that meets the needs of the target users. The aim is to ensure the system is well-planned, logically structured, and user-friendly. This report has been developed concurrently with the implementation phase, in accordance with the project guidelines

- Project Overview

The project is a prototype version of the Simply Rugby System application. The main goal of the application is to support coaches, who are usually the players parents, in effectively monitoring training information. The system allows the coach to log in, after which they see the main menu with basic information such as available teams and coaches. Additionally, they can choose the "My Team" option, which refers to the team the coach is assigned to. There, they have full permissions to modify parameters related to their players. They can also save the changes they made, and this data will automatically be loaded the next time the application is launched.

2. Business Model - Functional Analysis and Dynamic Design

- Identified Classes, Attributes and Methods

Based on additional documentation provided by the client, this section describes the functional requirements and structure of the Simply Rugby application. As with any use case diagram development process, we start from the beginning by focusing on the functionality of the prototype.

The system will be our rugby application.

Actors is someone or something that uses our system to achieve a goal or complete specific task. Actors are external objects and need to be placed outside the system.

A use case describes what our system does. Represents an action that accomplishes some sort of task within the system.

Relationships can be divided into four stages:

1. Association relationship and its basic communication or interaction.
2. Include a show dependency between a base use and an included use case. Every time the base use case is executed, the included use case is executed as well, or the base use case requires the included use case in order to be complete.
3. Extension will only happen if certain criteria are met, it's like extending the behaviour of our use case, like if we make a mistake in the login process, then show an error message.
4. Generalisation relationship is also known as inheritance. We can generalise a use case, but actors can also have it.

Lets define our four elements :

I. System

Simply Rugby application system

II. Actors

Coach

III. Use Cases

Login

Login Validation

Failed Login Attempt

View assigned team

Select team

View Players

Edit Player Skills

Save Player Skills

View All Coaches

Exit

IV. Relationship

Coach is related to:

Login

Login Validation

Failed Login Attempt

View assigned team

Select team

View Players

Edit Player Skills

Save Player Skills

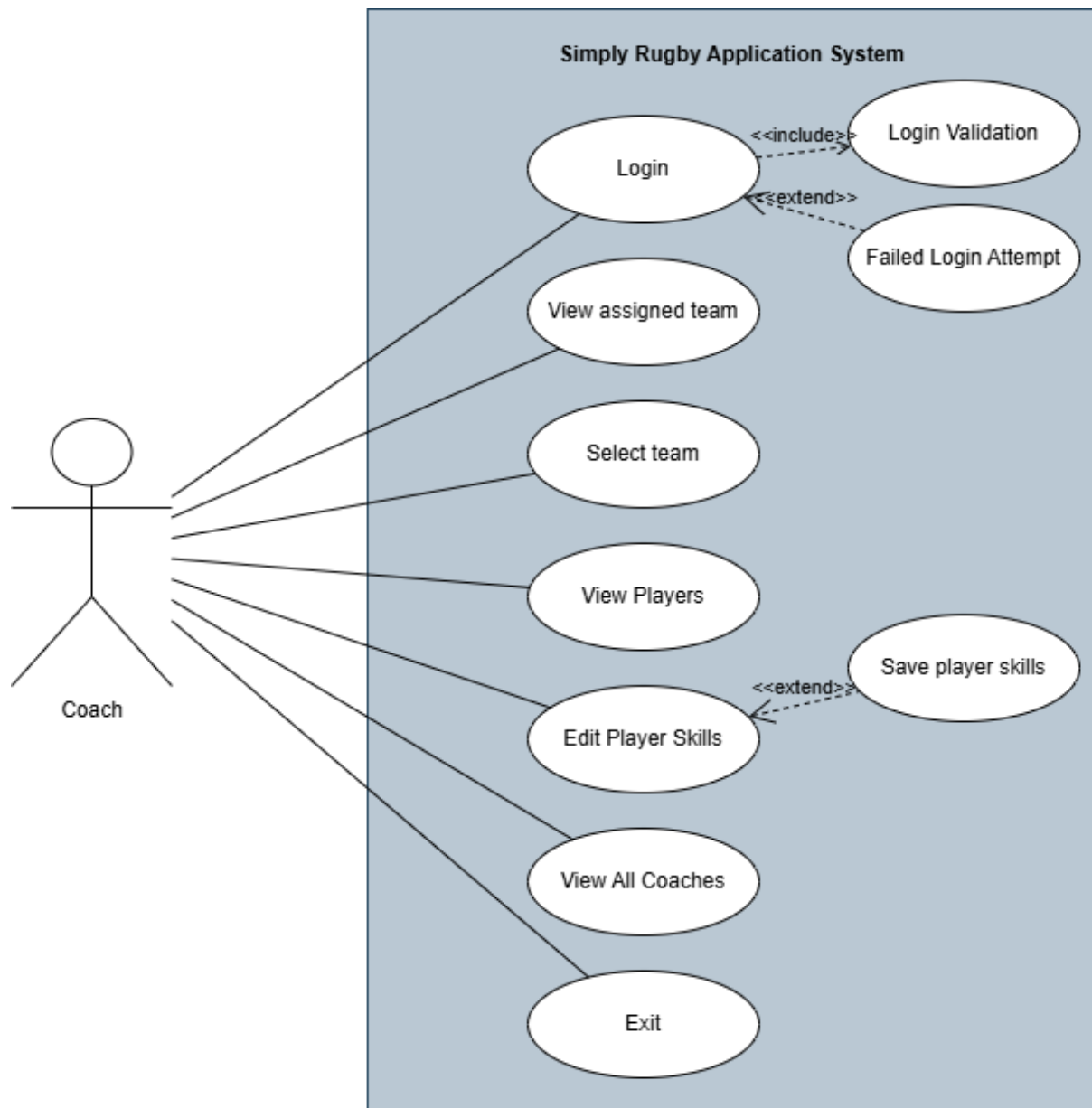
View All Coaches

Exit

Proposed Classes (with example attributes and methods)

Class Name	Attributes	Methods
Coach	username, password, firstName, secondName, List<Team> teams	getUsername(), getTeams(), toString()
Player	name, surname, position, address, email, skills, medical info	getSkills(), setSkillLevel(), toString()
Team	id, name, List<Player> players	addPlayer(), getPlayers(), toString()

- Use Case Diagram



The use case diagram illustrates how the actor Coach can interact with the system and what functionalities are available. It can be observed that the coach can log in, and the login process includes validation and returns a login result. Furthermore, the coach can view all teams and other coaches, select the team they are assigned to, and modify the parameters of the players. The UML Use Case Diagram was created to graphically represent user interactions with the system. Additionally, it helps in planning, analysing, and developing software.

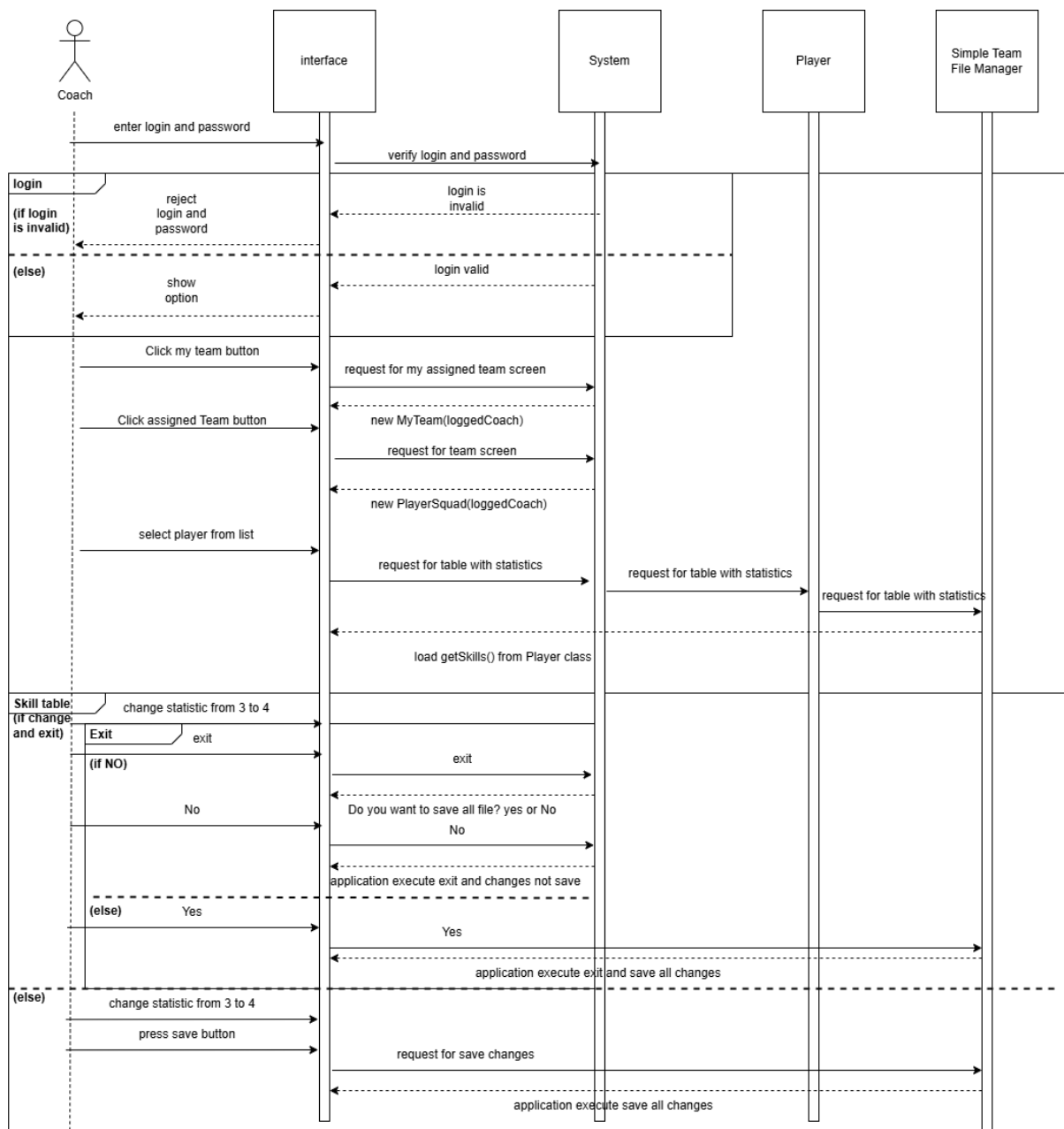
- Use Case Model 1 – "Login "

Use Case ID:	01
Use Case Name:	Login
Created by:	Kamil Milej
Date Created:	13/05/2025
Actors:	Coach
Trigger:	The coach tries to log in by entering their username and password.
Pre-conditions:	The coach must be registered in the Simply Rugby System.
Post-conditions:	Success State: The coach is logged in and authenticated. Failure State: The coach fails to log in due to invalid credentials.
Normal Flow:	1. The coach enters the username and password. 2. The coach clicks the "Log In" button. 3. The system validates the entered data (includes Validate Login). 4. If the username or password is incorrect, the system shows an error message (extends Failed Login Attempt). 5. If the username and password are valid, the system grants access to the main interface. 6. The user can perform further actions.
Extends (Alternative flows):	Failed Login Attempt
Includes:	Validate login

- Use Case Model 2 – "Edit Player Skills"

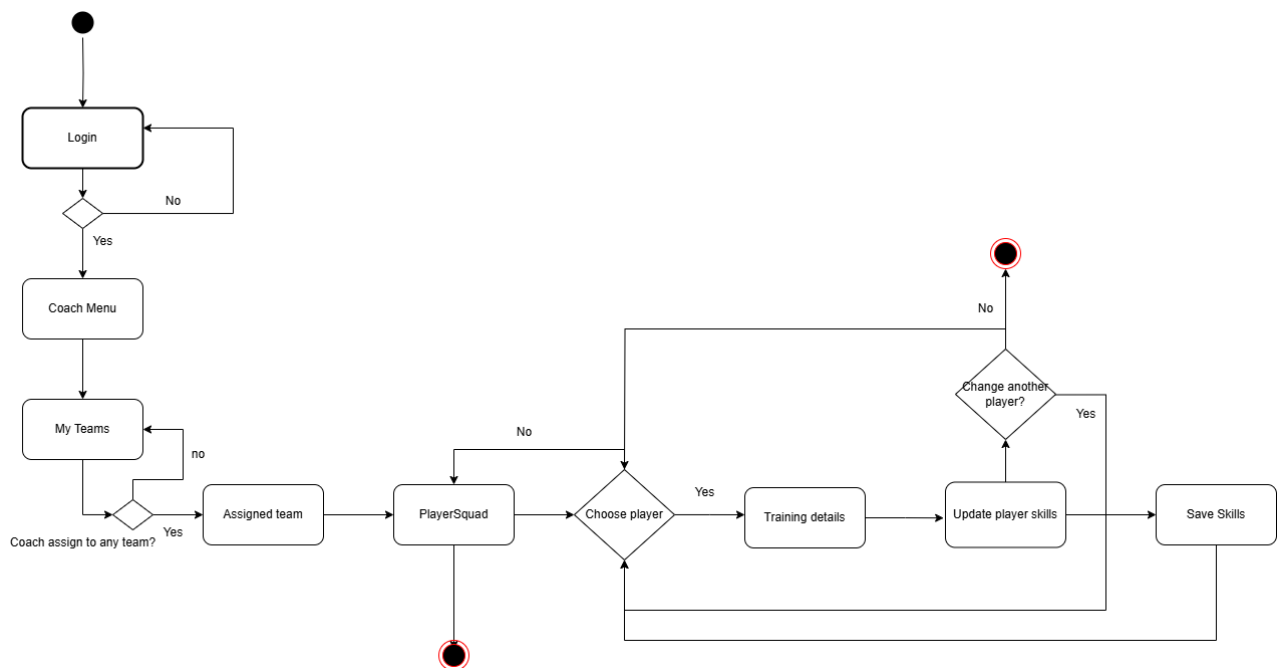
Use Case ID:	02
Use Case Name:	Edit Player Skills
Created by:	Kamil Milej
Date Created:	13/05/2025
Actors:	Coach
Trigger:	Coach selects a player from the assigned team and chooses to edit their skill levels.
Pre-conditions:	The coach must be logged in The coach must have access to at least one team with players
Post-conditions:	Success: Skill levels are updated in memory and displayed in the UI. Failure: No change is saved until the coach explicitly clicks "Save Skills".
Normal Flow:	1. Coach logs in and navigates to "My Team" 2. Coach selects a team and a player from the list. 3. The player's skill levels are displayed in a table 4. Coach edits skill levels using dropdowns 5. Changes are shown immediately in the UI but not persisted 6. Coach can now choose to save if want by press save button (exclude Save Player Skills) 7. Exit, before application will end ask to save all changes
Extends (Alternative flows):	Save Player Skills
Includes:	-

Sequence Diagram



The diagram includes interaction between the Coach, the interface, the system, the Player class, and the Simple Team File Manager. The process begins with the coach entering login credentials. Depending on the result of the login validation (valid or invalid), the coach either proceeds or receives a rejection message. After successful authentication, the coach operates through layers to view the assigned team and access player statistics. When a skill changes, the coach can choose to save or exit without saving. The system provides a confirmation prompt and performs the corresponding operation depending on the coach choice.

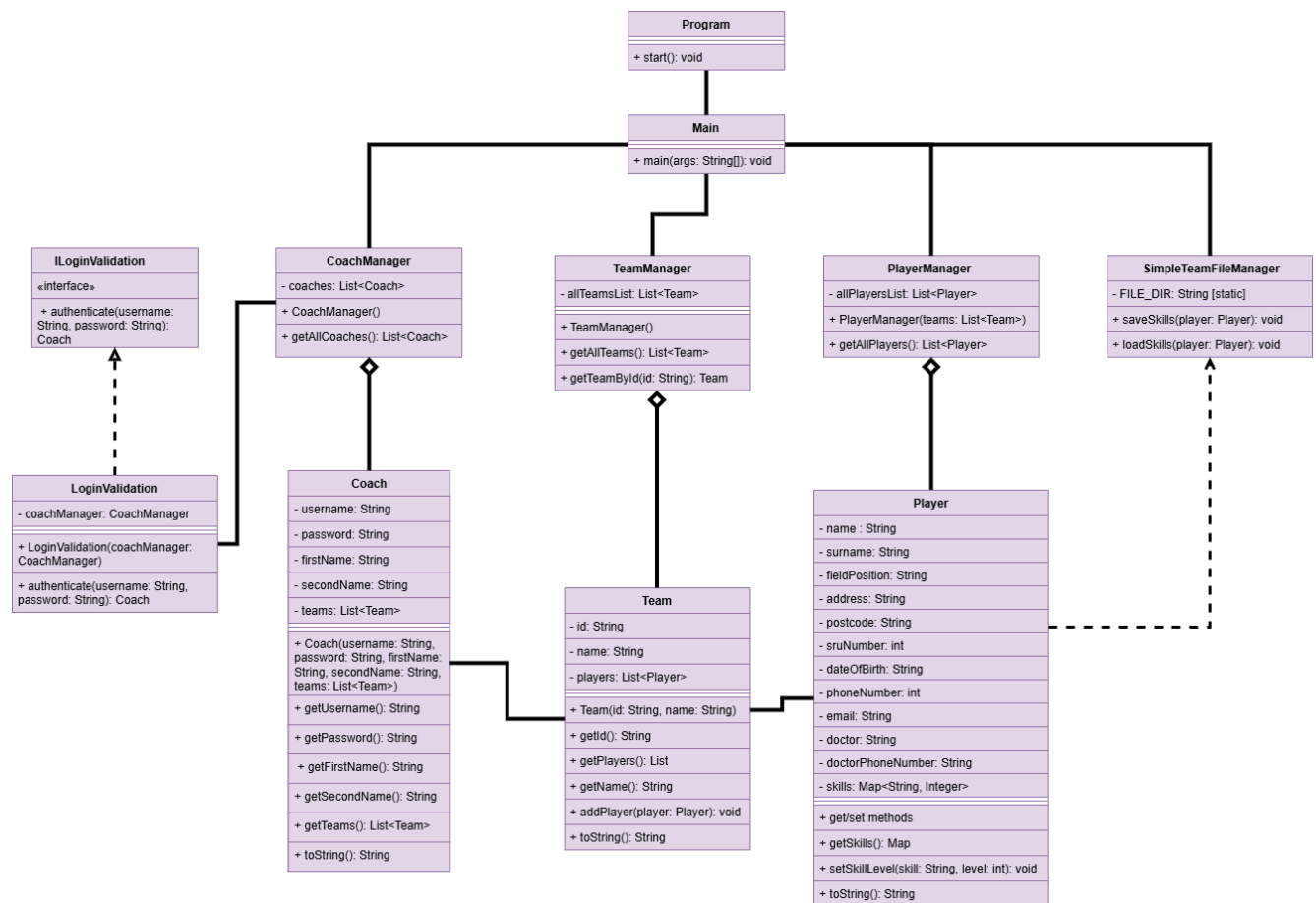
- Activity Diagram



The activity diagram above represents the logical workflow of the Coach user within the Simply Rugby application. The flow begins with the login process and proceeds through team and player selection, ending with all options of updating and saving player skills. After a successful login, the coach is directed to the main menu (CoachMenu), where the system checks whether they have been assigned to any team. If not, the only available option is to go back. If the coach is already assigned to a team, they can select the team, then choose a player and view their training data. From the PlayerSquad screen, the coach can select a player, which will display a panel with the player's statistics. The coach can save any changes by clicking the Save Skills button, switch to another player, and make further modifications if needed. After completing all actions, there is always the option to save the data and exit the application.

3. Static Model (UML)

- Class Diagram



The class diagram represents the static structure of the Simply Rugby application. It includes the main data model classes (Coach, Team, Player), management classes (CoachManager, TeamManager, PlayerManager), a class responsible for login validation logic (LoginValidation), and a utility class for reading and writing data (SimpleTeamFileManager).

The diagram also presents the Main class as the entry point of the application and the Program class, which runs the core logic.

Each class contains appropriate attributes and methods, following object-oriented programming principles. Diagram has the following relationships:

Aggregation – e.g. TeamManager contains a list of Team objects, PlayerManager contains a list of Player objects and CoachManager has list of Coaches.

Association – the Coach class is associated with a list of Teams, and each Team is associated with a list of Players.

Interface Implementation – LoginValidation implements the ILoginValidation interface, which separates the definition from the implementation and facilitates testing.

Static Methods - SimpleTeamFileManager provides static methods for saving and reading player data from a file.

The class diagram forms the foundation of the application's structure and helps illustrate how the different objects interact with each other. It has been designed with the principles of modularity, encapsulation, readability, and easy extensibility for future development stages.

The classes responsible for testing and the GUI (the visual layer of the application) are not included in this diagram, in accordance with the standard principles for creating this type of class diagram.

4. User and Interface Analysis

- User Analysis

The primary user of the Simply Rugby system is the Coach. This is the person responsible for leading a rugby team, managing players, their skills, and training progress. The coach uses the application to access player lists, edit player information, and record training progress.

The needs of a person using this application include:

Ease of use, meaning a simple and clear interface

Transparent presentation of all available options

Intuitive navigation

Reliable login functionality

The technical skills required to use the system should be basic—no knowledge of IT systems is necessary.

The system should be intuitive, meaning that navigating through the interface should be smooth and user-friendly.

There should be no unnecessary options that could in any way complicate usage.

The system should respond quickly, allowing fast updates and changes.

It should also be very easy to save all data and exit the application.

- HCI Design Considerations

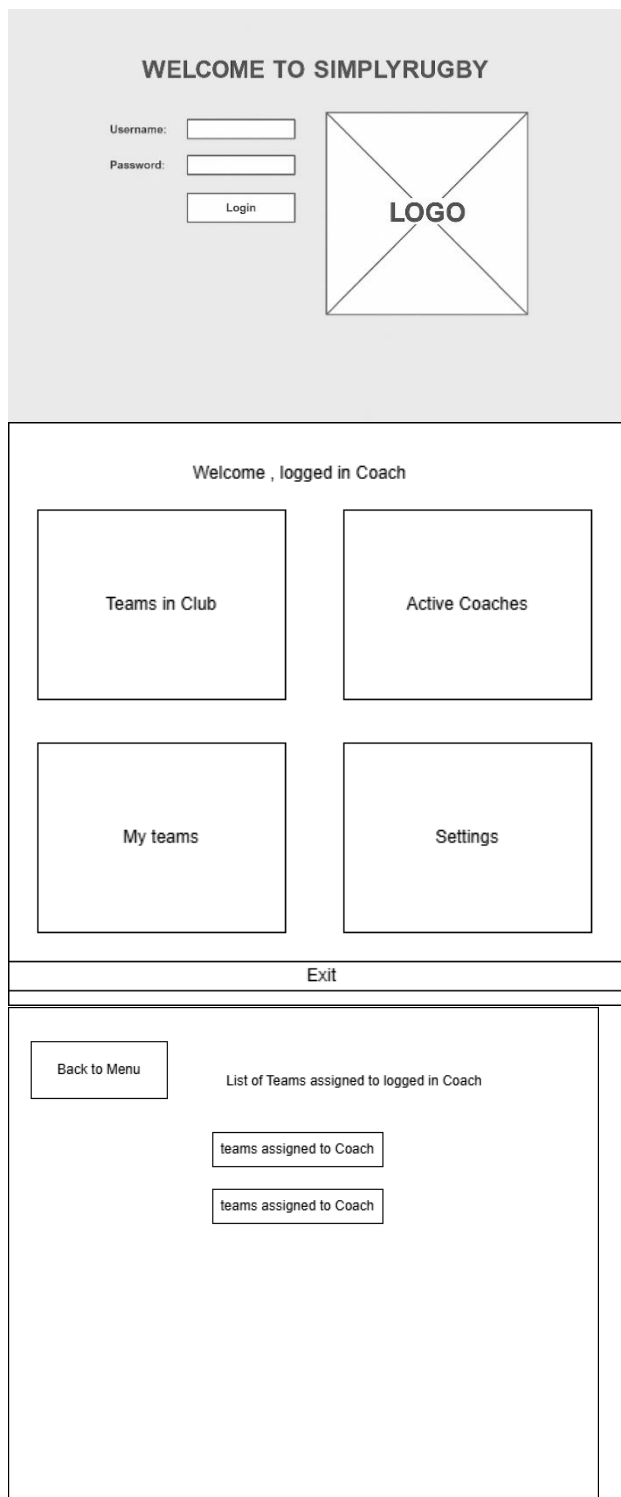
During the development of the Simply Rugby system, the interface was designed in accordance with key Human-Computer Interaction (HCI) principles to ensure that the system is perfectly tailored to the needs of every coach.

The main principles of HCI are:

simplicity	Not many buttons, big size, too not disturb user
consistency	All window has the same size and show in the same place, also buttons are in same place.
feedback	All necessary information, like what was chosen or picked, is shown
visibility of system status	System always show who is logged in and on which team work
user control	The user decides whether to save changes or exit without saving. The application does not force immediate actions and does not lock the user into lengthy processes.

5. User Interface Design

- Wireframes



The wireframe shows the login screen of the prototype. This is the first screen visible to the user upon launching the application. It includes two text fields - one for the username and one for the password - and a Login button.

This wireframe shows the main menu of the entire program with the core functionalities, represented by four large buttons and an Exit button.

In this window, we see a "Back to Menu" button, which allows the user to return to the main menu, as well as a label displaying who is currently logged in and the teams assigned to that user.

The wireframe shows a rectangular interface. At the top left is a button labeled 'Back to Menu'. To its right is the text 'Welcome logged in Coach'. Below this is the text 'Team chosen by coach : name and ID'. On the left side, there is a table with two rows: 'Finn Russell - Position' and 'Blair Kinghorn -Position'. Below this table is a larger box labeled 'List of Player assigned to this team'. To the right of the player list is a table with two columns, 'standard' and '2', and a row labeled 'Table with all statistics'. To the right of the statistics table is a button labeled 'Save skills'. At the bottom right is a button labeled 'Exit'.

This is the final screen, which also includes a "Back to Menu" button and a dynamic Welcome message displaying who is currently logged in and which team they belong to. Additionally, there is a List of Players when a player is selected, a table appears next to it where training data can be updated. After making the desired changes, the user can either save them by clicking Save Skills or exit the screen and choose to save before exiting.

6. Planning Summary and Next Steps

- Planning Stage Completion

All required models have been presented in this document, including:

- ✓ Use Case Diagram
- ✓ Two Use Case Models
- ✓ Sequence Diagram
- ✓ Activity Diagram
- ✓ Detailed Class Diagram
- ✓ Wireframe

The code was developed in parallel with the documentation (solution planning).

In Stage 3 – Evaluation, a formal report will be created, which will cover the entire project, including aspects related to the code.

- Pending Design Items

This report (Solution Planning) completes Stage 1.

To complete the entire project in accordance with the requirements of Simply Rugby, the following stages remain:

Testing, Documentation, and Evaluation.